

API Debugging — Worked Example: Solution

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)
- [Real cause vs. symptom fix](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Basic

Real cause: `max_attempts` and `timeout` are declared keyword-only (everything after the bare `*` in the signature). The caller passed `3` as a third *positional* argument, which Python won't accept — the function only takes 2 positional arguments (`method`, `url`).

The fix:

```
response = request_with_retry("GET", url, max_attempts=3)
```

The function's signature doesn't change — the caller does. Keyword-only arguments exist specifically so a call site can't accidentally pass the wrong value into the wrong slot by position.

Test that would have caught it:

```
def test_request_with_retry_accepts_keyword_max_attempts():
    response = request_with_retry("GET", "http://example.com", max_attempts=1)
    assert response is not None
```

Reading a `TypeError` about positional arguments should send you straight to the function's own definition, not to guessing about the network — this class of error happens before any request is even attempted.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Intermediate

Real cause: the error-handling code assumed a failed response's body is always JSON, and called `.json()` on it without checking first. When the final failed response is actually an HTML gateway error page (a common shape for a struggling upstream server), `.json()` itself throws — before the intended `TransientHTTPError` ever gets raised. The traceback shows this happening inside Doc04's `ask()` function, which is one layer removed from where the real bug lives (`http_client.py`) — reading from the bottom of the trace up gets you to the actual spot fast; reading from the top down (where `ask()` is) points at the wrong document.

The fix:

```
if attempt == max_attempts:
    try:
        error_body = response.json()
    except ValueError:
        error_body = response.text
    raise TransientHTTPError(response.status_code, error_body)
```

Building the error message itself has to survive a non-JSON body — it's exactly the "valid response, unexpected shape" case Doc02's Core Concepts calls out, just triggered on the *error path* instead of the happy path.

Test that would have caught it:

```
def test_transient_error_message_survives_non_json_body(fake_transport):
    fake_transport.always_returns(status=502, body="<html>Bad Gateway</html>")
    try:
        request_with_retry("GET", "http://example.com", max_attempts=1)
    except TransientHTTPError as error:
        assert error.status_code == 502
    else:
        assert False, "expected TransientHTTPError"
```

The test that would have caught this has to feed the wrapper a non-JSON error body on purpose — a fake transport that always returns clean JSON, even for errors, can't ever exercise this path.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Real-world

Real cause: `time.sleep(2 ** attempt)` has no randomness in it. Every caller that hits the same rate limit at the same moment computes the exact same sleep durations, so they all wake up and retry at the exact same instant, again and again — a thundering herd, colliding with the same limit on every single attempt instead of spreading out.

The fix:

```
import random

def backoff_seconds(attempt):
    base = 2 ** attempt
    jitter = random.uniform(0, base * 0.5)
    return base + jitter
```

Each caller now waits a slightly different amount, so a group that started retrying together spreads out instead of staying locked in step.

Test that would have caught it:

```
def test_backoff_seconds_is_not_identical_across_callers():
    values = []
    for _ in range(20):
        values.append(backoff_seconds(attempt=2))
    assert len(set(values)) > 1
```

A test running one caller in isolation can't catch this at all — the bug is specifically about what happens when *multiple* callers retry at the same time, so the test has to simulate that, not just check one call's delay in isolation.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Multi-agent

Real cause: a cap on attempt *count* doesn't bound total elapsed time when backoff keeps growing — 5 attempts at exponential backoff can still add up to 30+ seconds of silent waiting, inside a single tool call the graph-level log has no visibility into. From the Supervisor's log alone, that looks exactly like a stuck graph, which sends a debugging session down the wrong layer entirely (checking LangGraph edges and state, when the real problem is one layer lower, inside the HTTP retry wrapper).

The fix:

```
import time

def request_with_retry(method, url, *, max_attempts=3, timeout=(3, 5), max_total_
    start_time = time.monotonic()
    for attempt in range(1, max_attempts + 1):
        if time.monotonic() - start_time > max_total_seconds:
            raise TransientHTTPError(None, "exceeded max_total_seconds before suc
        try:
            return send_request(method, url, timeout=timeout)
        except (TimeoutError, RetryableStatusError):
            if attempt == max_attempts:
                raise
            time.sleep(backoff_seconds(attempt))
```

A hard ceiling on total wall-clock time, checked before every attempt, turns a silent multi-attempt hang into a fast, clear failure — and, just as important, logs it as an HTTP-layer failure, not an unexplained gap in the Supervisor's log.

Test that would have caught it:

```
def test_request_with_retry_gives_up_after_max_total_seconds(fake_transport, fake
    fake_transport.always_times_out()
    try:
        request_with_retry("GET", "http://example.com", max_attempts=10, max_tota
    except TransientHTTPError:
        pass
    else:
```

```
assert False, "expected TransientHTTPError"  
assert fake_clock.elapsed_seconds() <= 5
```

This is the kind of failure that a full-pipeline run finds but an isolated `request_with_retry()` unit test using a fake transport (instant, no real sleeping) never would — unless the test deliberately checks elapsed time against a real or simulated clock, the way this one does.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Real cause vs. symptom fix

A symptom fix at any of these rounds is tempting because it's fast: wrap the call in a broader `try/except` and return `None` on any failure, or just bump `max_attempts` up until the flaky test stops flaking. Both make today's specific failure stop showing up without touching why it happens — a swallowed exception just moves the missing data problem one layer downstream, to whatever code trusted a `None` it never expected, and a higher retry count just makes a thundering herd retry more times before colliding again. The real-cause fixes above all share one shape: they make the *actual* failure condition (a non-JSON error body, synchronized retries, unbounded total wait time) structurally impossible, not just less likely to show up in your next test run — which is exactly the "checking layer by layer" habit Core Concepts asks for: the API layer's own failure-handling code can have its own bugs, separate from whatever layer is calling it.